

DevOps Assessment – Continuous Integration with GitHub Actions

Objective

In this assessment, you will implement **Continuous Integration (CI)** for a Python application using **GitHub Actions**.

You will work directly in a GitHub repository and use GitHub Codespaces where required.

Your completed repository will be assessed based on the implementation, workflow configuration, commits, and successful execution of the GitHub Actions workflows.

Part 1: Run and Assess the Python Application

Task 1: Set Up the Development Environment

1. Open the provided [GitHub repository](#) and create a **GitHub Codespace** using the default configuration.
2. Confirm that you are working in **your copy of the exercise repository**, not the original repository.
3. Wait for Visual Studio Code to load.
4. Open the following files from the project:
 - src/calculations.py
 - tests/calculation_tests.py
5. Review the application code and existing tests to understand what the application does and how it is currently being tested.
6. Open the integrated terminal in the Codespace.
7. Create a Python virtual environment for the project.
8. Activate the virtual environment.
9. Install the dependencies specified by the project.
10. Install the Python testing and code-coverage tools required to execute the tests and generate a coverage report.

Requirement: Do not modify the existing application or test files unless specifically instructed to do so in a task.

Task 2: Execute the Test Suite

1. Execute the complete unit-test suite for the application.
2. Generate a code-coverage report for the src directory.
3. Configure the test execution so that sufficient test execution details are displayed in the terminal.
4. Review the resulting test and coverage information.
5. Identify whether the existing tests provide complete coverage of the application.
6. Add a comment to the relevant GitHub issue indicating that you have completed the coverage analysis and are ready for the next step.

Part 2: Configure Continuous Integration

Task 3: Create a GitHub Actions Test Workflow

Create a GitHub Actions workflow that automatically executes the Python test suite. Your workflow must satisfy the following requirements:

Trigger

Configure the workflow so that it runs when a **pull request targets the main branch**.

The workflow should not rely on a push event for this assessment.

Test Job

Configure a job that:

1. Runs on an appropriate GitHub-hosted Linux runner.
2. Retrieves the repository contents into the runner environment.
3. Sets up the required Python environment.
4. Installs the project's dependencies.
5. Installs the required testing tools.
6. Executes the existing Python tests.
7. Displays detailed test information in the workflow logs.

Repository Requirements

- Store the workflow in the appropriate GitHub Actions workflow directory.
- Use an appropriate workflow filename.
- Follow valid YAML syntax and indentation.
- Use appropriate GitHub Actions provided by the Actions ecosystem where required.

Commit Requirement

Commit the completed workflow directly to the main branch.

After committing:

1. Confirm that the workflow file exists in the repository.
2. Open the **Actions** tab.
3. Verify that GitHub recognizes the workflow.
4. Inspect the workflow configuration and execution status.

Part 3: Configure Test Coverage Automation

Task 4: Create a Coverage Workflow

Create a **second GitHub Actions workflow** dedicated to Python test coverage.

The workflow must be stored in the `.github/workflows/` directory.

Use the following required filename:

`python-coverage.yml`

Workflow Trigger

Configure the workflow to execute when a pull request targets the main branch.

Permissions

Configure the workflow with the permissions necessary for it to publish coverage information to the relevant pull request.

Coverage Job

Create a dedicated job for the coverage process.

The job must:

1. Run on an appropriate GitHub-hosted Linux runner.
2. Retrieve the repository contents.
3. Set up the required Python version.
4. Install the project dependencies.
5. Install the required testing and coverage packages.
6. Execute the project's tests while generating coverage information for the src directory.
7. Publish the resulting coverage information as a comment on the pull request using an appropriate pre-built GitHub Action.
8. Enforce a minimum test-coverage requirement.
9. Cause the workflow to fail when the required minimum coverage level is not achieved.

Workflow Configuration

Your workflow should use appropriate:

- workflow-level configuration
- job configuration
- runner configuration
- steps
- GitHub Actions
- package installation commands
- test commands
- coverage configuration
- GitHub token configuration

Do not modify the application source code or existing tests to artificially increase the coverage result.

Part 4: Commit and Push Your Implementation

Task 5: Save Your Work

After completing both workflows:

1. Save all changes in your Codespace.
2. Review the files you have created or modified.
3. Commit your changes with a meaningful commit message.
4. Push the changes to the GitHub repository.
5. Confirm that the latest changes are visible on GitHub.

Part 5: Validate the CI Implementation

Task 6: Validate Workflow Execution

Use GitHub to verify that your implementation works as intended.

You must verify:

1. The test workflow is recognized by GitHub Actions.

2. The coverage workflow is recognized by GitHub Actions.
3. The workflows respond to the required pull-request event.
4. The Python environment is successfully configured by the workflows.
5. Dependencies are successfully installed.
6. The automated tests execute successfully.
7. Coverage information is generated.
8. Coverage information is published to the pull request.
9. The minimum coverage requirement is enforced.
10. Workflow results are visible in GitHub Actions.

If a workflow fails, investigate the failure and correct your implementation before submitting the assessment.

Submission Requirements

Before submitting, ensure that your GitHub repository contains:

- The original Python application.
- The original test suite.
- Your completed GitHub Actions test workflow.
- `python-coverage.yml`.
- All required workflow configuration.
- Your commits containing the completed implementation.

Final Verification

Before submission:

- Ensure all required files have been committed.
- Ensure all changes have been pushed to GitHub.
- Ensure the workflows have been executed.
- Ensure the latest workflow runs have completed.
- Ensure there are no unresolved YAML or configuration errors.
- Ensure the repository reflects your final implementation.

Submission Evidence

Submit the following:

1. **GitHub repository URL**
2. **Pull request URL used to validate the workflows**
3. **Screenshot showing the GitHub Actions workflows**
4. **Screenshot showing a successful test workflow execution**
5. **Screenshot showing the coverage result/comment on the pull request**
6. **Screenshot showing the final repository/workflow files**

Your repository and submitted evidence must represent the **final version of your implementation**.

Important Instructions

- Complete the assessment independently.
- Do not modify the application logic to manipulate the assessment results.
- Do not artificially increase test coverage.
- Do not remove or bypass existing tests.
- Do not disable workflow checks to obtain a successful result.
- Use valid YAML syntax and maintain correct indentation.
- All required implementation must be committed and pushed to your GitHub repository.
- The final GitHub repository will be used as the primary source for assessment.